C# Features Reference Guide

Introduction

This guide overviews fundamental C# features essential for .NET development. It focuses on object-oriented programming (OOP) principles, control structures, and class members that form the backbone of creating organized, reusable, and secure code in C#.

Key Concepts

Object-Oriented Programming (OOP) in C#

- Classes and objects: Classes serve as blueprints for creating objects, defining their attributes and behaviors.
- Inheritance: Enables code reuse by allowing classes to inherit characteristics from other classes, establishing a hierarchy.
- Polymorphism: Allows methods to perform differently depending on the parameters used when calling the method, promoting code flexibility.
- Encapsulation: Using access modifiers (`public`, `private`, `protected`) to manage data visibility and accessibility, enhancing security and structure.

Class Members

- Fields and properties: Fields hold private data; properties provide controlled access to this data, supporting encapsulation.
- Methods: Define actions associated with a class. Static methods apply to the class itself, while instance methods operate on specific class instances.
- Abstract and virtual methods: Abstract methods require implementation in derived classes; virtual methods have a default base class implementation yet can be overridden.

Control Structures

C# includes basic control structures that manage applications' logical flow and decision-making.

- If-Else Statements: Control the flow of logic by executing different blocks of code based on specified conditions.

- Loops: Repeatedly execute a block of code as long as a given condition is met. Common loop types include for, while, do-while, and foreach.
- Switch Cases: Allow branching logic by matching a variable's value to specific cases, executing corresponding code.

Conclusion

Understanding these essential C# features allows developers to build robust, maintainable, scalable applications. By focusing on OOP principles and structured code, C# developers can create solutions that are both flexible and efficient in the .NET environment.